

```

selector:
  matchLabels:
    app: taco-cloud
template:
  metadata:
    labels:
      app: taco-cloud
  spec:
    containers:
      - name: taco-cloud-container
        image: tacocloud/tacocloud:latest
        livenessProbe:
          initialDelaySeconds: 2
          periodSeconds: 5
          httpGet:
            path: /actuator/health/liveness
            port: 8080
        readinessProbe:
          initialDelaySeconds: 2
          periodSeconds: 5
          httpGet:
            path: /actuator/health/readiness
            port: 8080

```

This tells Kubernetes, for each probe, to make a GET request to the given path on port 8080 to get the liveness or readiness status. As configured here, the first request should happen 2 seconds after the application pod is running and every 5 seconds thereafter.

MANAGING LIVENESS AND READINESS

How do the liveness and readiness statuses get set? Internally, Spring itself or some library that the application depends on can set the statuses by publishing an availability change event. But that ability isn't limited to Spring and its libraries; you can also write code in your application that publishes these events.

For example, suppose that you want to delay the readiness of your application until some initialization has taken place. Early on in the application lifecycle, perhaps in an `ApplicationRunner` or `CommandLineRunner` bean, you can publish a readiness state to refuse traffic like this:

```

@Bean
public ApplicationRunner disableLiveness(ApplicationContext context) {
    return args -> {
        AvailabilityChangeEvent.publish(context,
            ReadinessState.REFUSING_TRAFFIC);
    };
}

```

Here, the `ApplicationRunner` is given an instance of the Spring application context as a parameter to the `@Bean` method. This is necessary because the static `publish()`

method needs it to publish the event. Once initialization is complete, the application's readiness state can be updated to accept traffic in a similar way, as shown next:

```
AvailabilityChangeEvent.publish(context, ReadinessState.ACCEPTING_TRAFFIC);
```

Liveness status can be updated in very much the same way. The key difference is that instead of publishing `ReadinessState.ACCEPTING_TRAFFIC` or `ReadinessState.REFUSING_TRAFFIC`, you'll publish `LivenessState.CORRECT` or `LivenessState.BROKEN`. For example, if in your application code you detect an unrecoverable fatal error, your application can request that it be killed and restarted by publishing `Liveness.BROKEN` like this:

```
AvailabilityChangeEvent.publish(context, LivenessState.BROKEN);
```

Shortly after this event is published, the liveness endpoint will indicate that the application is down, and Kubernetes will take action by restarting the application. This gives you very little time to publish a `LivenessState.CORRECT` event. But if you determine that, in fact, the application is healthy after all, then you can undo the broken event by publishing a new event like this:

```
AvailabilityChangeEvent.publish(context, LivenessState.CORRECT);
```

As long as Kubernetes hasn't hit your liveness endpoint since you set the status to broken, your application can chalk this up as a close call and keep serving requests.

18.4 Building and deploying WAR files

Throughout the course of this book, as you've developed the applications that make up the Taco Cloud application, you've run them either in the IDE or from the command line as an executable JAR file. In either case, an embedded Tomcat server (or Netty, in the case of Spring WebFlux applications) has always been there to serve requests to the application.

Thanks in large part to Spring Boot autoconfiguration, you've been spared from having to create a `web.xml` file or servlet initializer class to declare Spring's `DispatcherServlet` for Spring MVC. But if you're going to deploy the application to a Java application server, you're going to need to build a WAR file. And, so that the application server will know how to run the application, you'll also need to include a servlet initializer in that WAR file to play the part of a `web.xml` file and declare `DispatcherServlet`.

As it turns out, building a Spring Boot application into a WAR file isn't all that difficult. In fact, if you chose the WAR option when creating the application through the Initializr, then there's nothing more you need to do.

The Initializr ensures that the generated project will contain a servlet initializer class, and the build file will be geared to produce a WAR file. If, however, you chose to build a JAR file from the Initializr (or if you're curious as to what the pertinent differences are), then read on.

First, you'll need a way to configure Spring's `DispatcherServlet`. Although this could be done with a `web.xml` file, Spring Boot makes this even easier with `SpringBootServletInitializer`. `SpringBootServletInitializer` is a special Spring Boot-aware implementation of Spring's `WebApplicationInitializer`. Aside from configuring Spring's `DispatcherServlet`, `SpringBootServletInitializer` also looks for any beans in the Spring application context that are of type `Filter`, `Servlet`, or `ServletContextInitializer` and binds them to the servlet container.

To use `SpringBootServletInitializer`, create a subclass and override the `configure()` method to specify the Spring configuration class. The next code listing shows `TacoCloudServletInitializer`, a subclass of `SpringBootServletInitializer` that you'll use for the Taco Cloud application.

Listing 18.1 Enabling Spring web applications via Java

```
package tacos;

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.context.web.SpringBootServletInitializer;

public class TacoCloudServletInitializer
    extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(
        SpringApplicationBuilder builder) {
        return builder.sources(TacoCloudApplication.class);
    }
}
```

As you can see, the `configure()` method is given a `SpringApplicationBuilder` as a parameter and returns it as a result. In between, it calls the `sources()` method that registers Spring configuration classes. In this case, it registers only the `TacoCloudApplication` class, which serves the dual purpose of a bootstrap class (for executable JARs) and a Spring configuration class.

Even though the application has other Spring configuration classes, it's not necessary to register them all with the `sources()` method. The `TacoCloudApplication` class, annotated with `@SpringBootApplication`, implicitly enables component scanning. Component scanning discovers and pulls in any other configuration classes that it finds.

For the most part, `SpringBootServletInitializer`'s subclass is boilerplate. It references the application's main configuration class. But aside from that, it'll be the same for every application where you'll be building a WAR file. And you'll almost never need to make any changes to it.

Now that you've written a servlet initializer class, you must make a few small changes to the project build. If you're building with Maven, the change required is as simple as ensuring that the `<packaging>` element in `pom.xml` is set to `war`, as shown here:

```
<packaging>war</packaging>
```

The changes required for a Gradle build are similarly straightforward. You must apply the war plugin in the build.gradle file as follows:

```
apply plugin: 'war'
```

Now you're ready to build the application. With Maven, you'll use the Maven wrapper script that the Initializr used to execute the package goal like so:

```
$ mvnw package
```

If the build is successful, then the WAR file can be found in the target directory. On the other hand, if you were using Gradle to build the project, you'd use the Gradle wrapper to execute the build task as follows:

```
$ gradlew build
```

Once the build completes, the WAR file will be in the build/libs directory. All that's left is to deploy the application. The deployment procedure varies across application servers, so consult the documentation for your application server's specific deployment procedure.

It may be interesting to note that although you've built a WAR file suitable for deployment to any Servlet 3.0 (or higher) servlet container, the WAR file can still be executed at the command line as if it were an executable JAR file as follows:

```
$ java -jar target/taco-cloud-0.0.19-SNAPSHOT.war
```

In effect, you get two deployment options out of a single deployment artifact!

18.5 The end is where we begin

Over the past several hundred pages, we've gone from a simple start—or start.spring.io, more specifically—to deploying an application in the cloud. I hope that you've had as much fun working through these pages as I've had writing them.

But while this book must come to an end, your Spring adventure is just beginning. Using what you've learned in these pages, go build something amazing with Spring. I can't wait to see what you come up with!

Summary

- Spring applications can be deployed in a number of different environments, including traditional application servers and PaaS environments like Cloud Foundry, or as Docker containers.
- Building as an executable JAR file allows a Spring Boot application to be deployed to several cloud platforms without the overhead of a WAR file.

- When building a WAR file, you should include a class that subclasses `SpringBootServletInitializer` to ensure that Spring's `DispatcherServlet` is properly configured.
- Containerizing Spring applications is as simple as using the Spring Boot build plugin's support for building images. These images can then be deployed anywhere Docker containers can be deployed, including in Kubernetes clusters.

appendix

Bootstrapping Spring applications

You can kick-start your Spring projects in a lot of ways, and which you choose is largely a matter of personal taste. Many of the choices will be decided by which IDE is your favorite.

All but one of these options are based on the Spring Initializr, which is a REST API that generates Spring Boot projects for you. The various IDE choices are nothing more than clients for that REST API. Additionally, you have a few ways to use the Spring Initializr API outside of your IDE. This appendix takes a quick look at all of these options.

A.1 *Initializing a project with Spring Tool Suite*

To initialize a new Spring project with Spring Tool Suite, choose the Spring Starter Project menu option from the File > New menu, as shown in figure A.1.

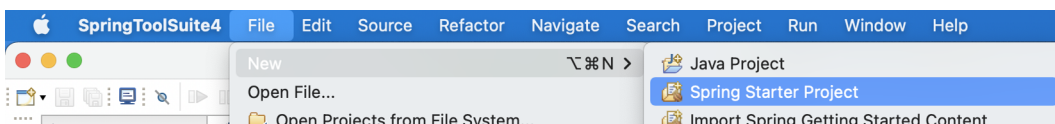


Figure A.1 Starting a new project in Spring Tool Suite

NOTE This is an abbreviated description of using Spring Tool Suite to initialize a Spring project. For a more detailed explanation, see section 1.2.1.

You'll be shown the first page of the project creation dialog box (figure A.2). On this page, you'll define basic project information, such as the project's name, coordinates

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

Figure A.2 Defining basic project information

(group ID and artifact ID), version, and base package name. You can also specify whether the project will be built with Maven or Gradle, whether the build will produce a JAR file or a WAR file, which version of Java to build with, and even an alternate JVM language to use, such as Groovy or Kotlin.

The first field on this page asks you to specify the location of the Spring Initializr service. If you're running or using a custom instance of the Initializr, you'll want to specify the base URL of the Initializr service here. Otherwise, you'll be fine leaving it with the default that points to <http://start.spring.io>.

After you've defined the basic project information, click Next to see the project dependencies page (see figure A.3).

New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

- ☐ Codecentric's Spring Boot Actuator
- ☐ Codecentric's Spring Boot Actuator
- ☐ H2 Database
- ☒ Lombok
- ☐ Spring Boot Actuator
- ☐ Spring Configuration Processor
- ☐ Spring Data JPA
- ☐ Spring Reactive Web
- ☒ Spring Web

Available:

- ▶ Developer Tools
- ▶ Google Cloud Platform
- ▶ I/O
- ▶ Messaging
- ▶ Microsoft Azure
- ▶ NoSQL
- ▶ Observability
- ▶ Ops
- ▶ SQL
- ▶ Security
- ▶ Spring Cloud
- ▶ Spring Cloud Circuit Breaker
- ▶ Spring Cloud Config
- ▶ Spring Cloud Discovery
- ▶ Spring Cloud Messaging
- ▶ Spring Cloud Routing
- ▶ Spring Cloud Tools

Selected:

- X Spring Boot DevTools
- X Lombok
- X Thymeleaf
- X Spring Web

Make Default Clear Selection

< Back Next > Cancel Finish

Figure A.3 Specifying project dependencies

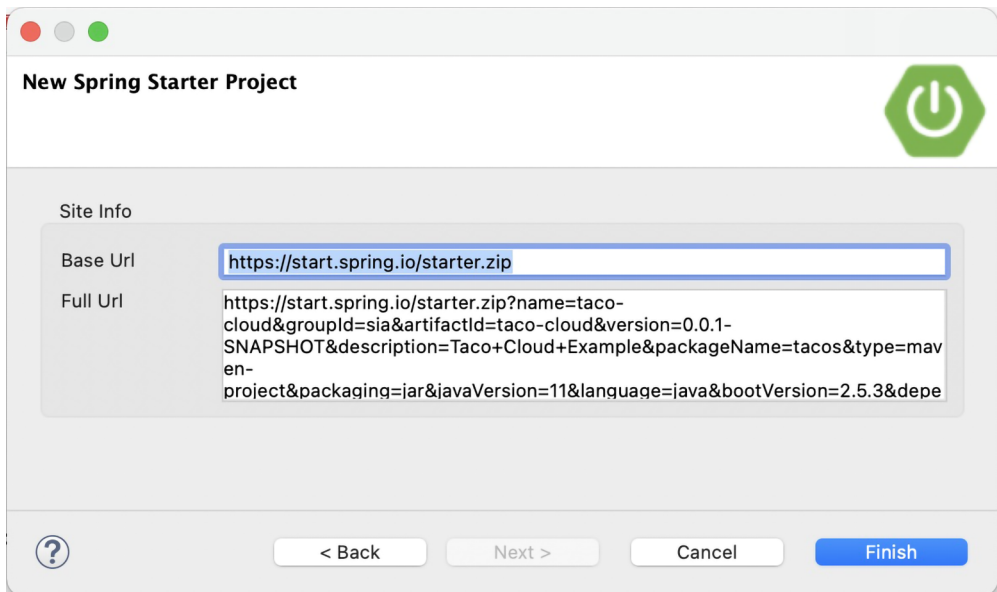
On the project dependencies page, you can specify all of the dependencies your project will need. Many of these dependencies are Spring Boot Starter dependencies, although some other dependencies are commonly used in Spring projects.

The available dependencies are listed on the left side, organized in groups that can be expanded or collapsed. If you're having trouble finding a dependency, you can also search for dependencies to narrow down your choices.

To add a dependency to the generated project, select the check box next to the dependency name. Your selections will appear in the list on the right side under the Selected header. You can remove a dependency by clicking the X next to the selected dependency, or click Clear Selection to remove all selected dependencies.

As an added convenience, if you find that you have a certain core set of dependencies that you always (or often) use for your projects, you can click the Make Default button after selecting those dependencies, and they'll already be selected the next time you create a project.

After making your selections, click Finish to generate the project and add it to your workspace. If, however, you want to use an Initializr other than the one at <http://start.spring.io>, click Next to set the Initializr base URL, as shown in figure A.4.



New Spring Starter Project

Site Info

Base Url:

Full Url: `https://start.spring.io/starter.zip?name=taco-cloud&groupId=sia&artifactId=taco-cloud&version=0.0.1-SNAPSHOT&description=Taco+Cloud+Example&packageName=tacos&type=maven-project&packaging=jar&javaVersion=11&language=java&bootVersion=2.5.3&depe`



Figure A.4 Optionally specifying the Initializr base URL

The Base Url field specifies the URL where the Initializr API is listening. This is the only field you can change on this page. The Full Url field shows the complete URL that will be used to request a new project from the Initializr.

A.2 Initializing a project with IntelliJ IDEA

To get started on a new Spring project in IntelliJ IDEA, choose the Project menu item from the File > New menu, as shown in figure A.5.

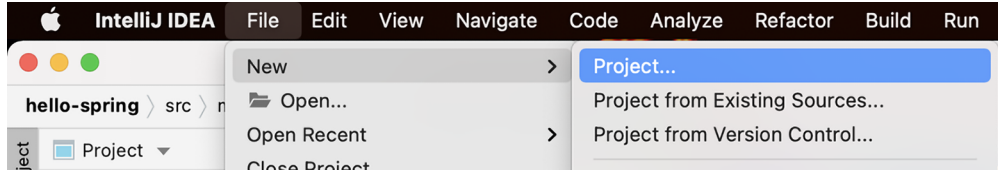


Figure A.5 Starting a new Spring project in IntelliJ IDEA

This opens up the first page of a new Spring Initializr project wizard. You'll be presented with a page that asks for essential project information, as shown in figure A.6.

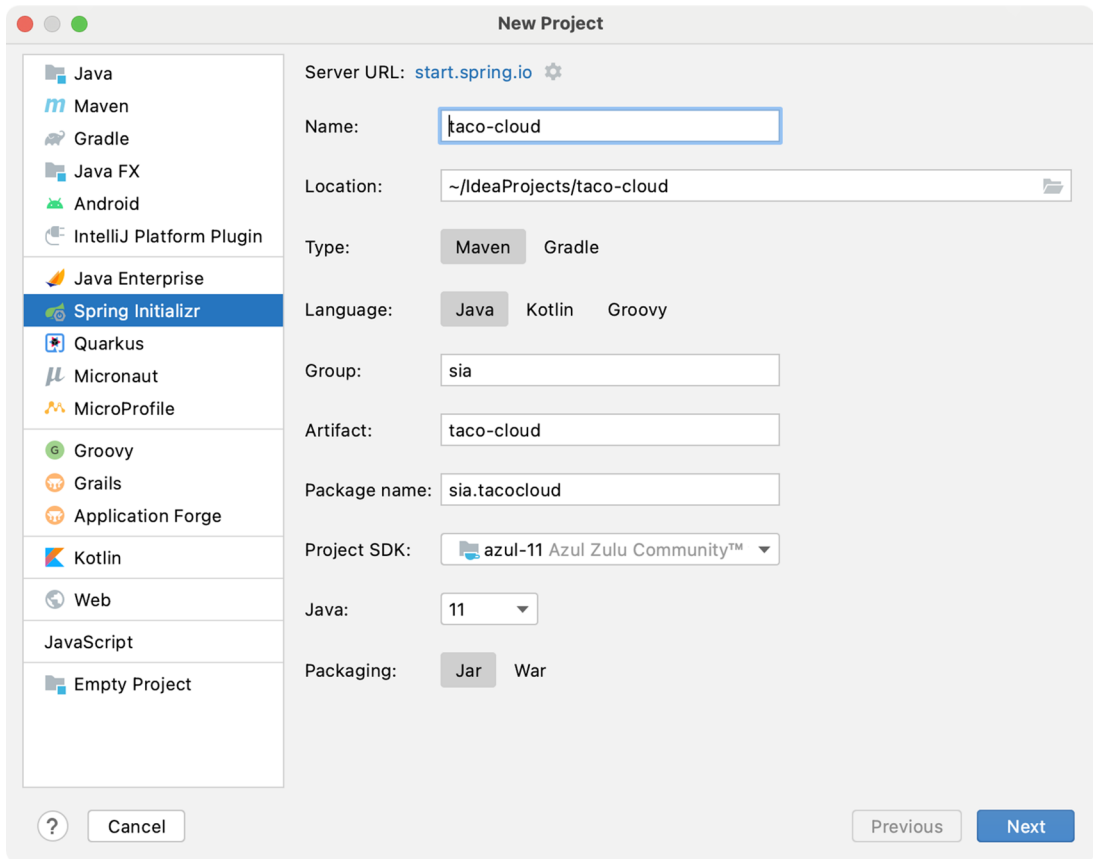


Figure A.6 Specifying essential project information in IntelliJ IDEA

You may recognize some of the fields on this page as information that might appear in a Maven `pom.xml` file—in fact, if you select Maven Project from the Type field, that's exactly how it will be used. You're welcome to choose Gradle Project instead if Gradle is your preference.

Once you've filled in the essential project information, click Next to be shown the project dependencies page (see figure A.7).

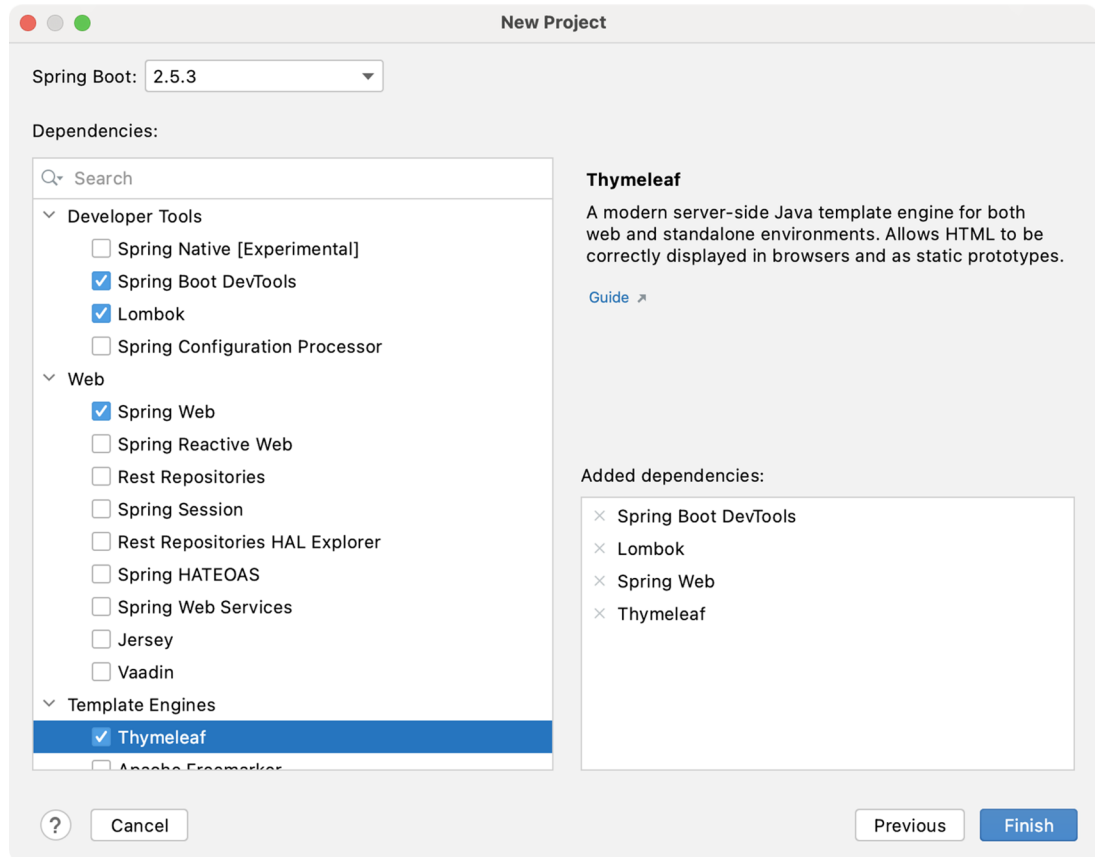


Figure A.7 Selecting project dependencies

The dependencies are organized by category in the far-left list. Selecting a category will result in that category's options being presented in the middle list. Your selected dependencies will be listed (according to category) in the right list.

After all of your dependencies have been selected, click Finish. Your project will be created and loaded into the IntelliJ IDEA workspace.

A.3 Initializing a project with NetBeans

Before you can create a new Spring Boot project in NetBeans, you need to install a plugin that enables Spring Boot development in NetBeans. The NB Spring Boot plugin adds features to NetBeans that are similar to those built into Spring ToolSuite and IntelliJ IDEA.

To install the plugin, select the Plugins option from the Tools menu, as shown in figure A.8.

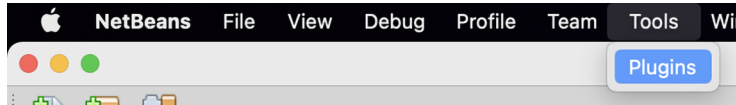


Figure A.8 The NetBeans Plugins menu item

You'll be shown a list of available plugins for NetBeans, including the NB Spring Boot plugin, as shown in figure A.9.

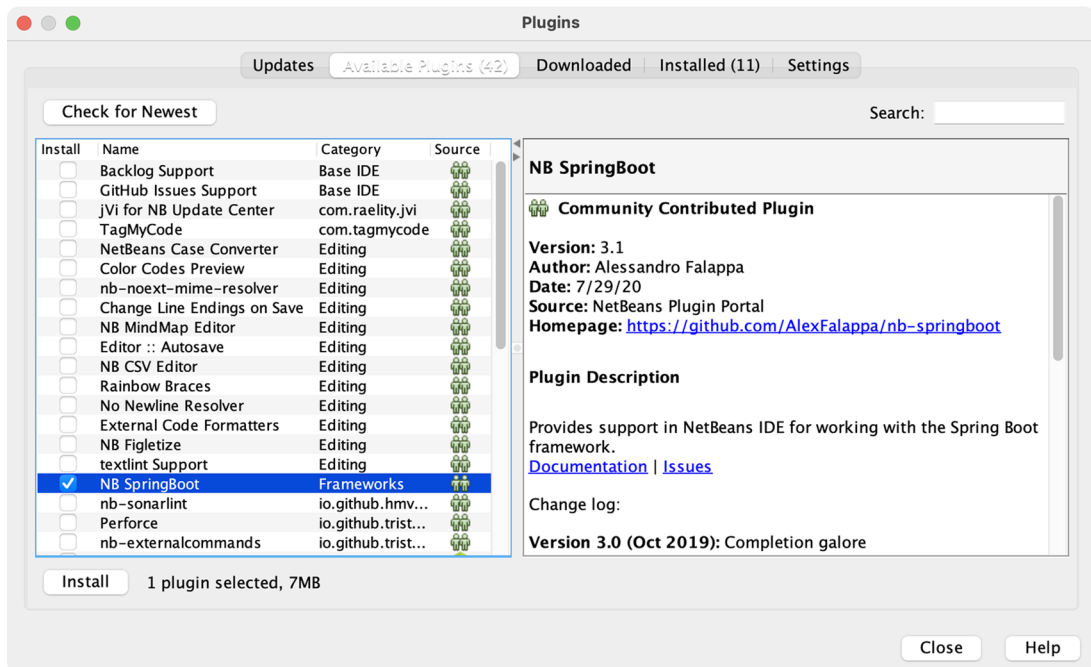


Figure A.9 Selecting the NB Spring Boot plugin

Click Install to begin the installation of the Spring Boot plugin. You'll be prompted with a handful of dialogs to confirm your decision and to acknowledge the plugin license agreement. Simply click Next through them all until you get to the last one, then click Install. Finally, you'll be prompted to restart NetBeans for the plugin to take effect.

After installing the Spring Boot plugin, you are ready to initialize a new Spring Boot project in NetBeans. To create a new Spring project in NetBeans, start by selecting the New Project menu item under the File menu, as shown in figure A.10.

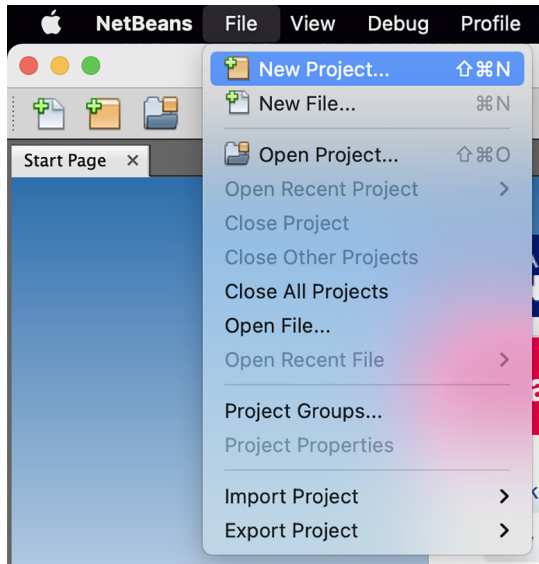


Figure A.10 Starting a new Spring project in NetBeans

You'll be shown the first page of the new project wizard. As shown in figure A.11, this page will let you choose what kind of project you want to create.

For a Spring Boot project, select Java with Maven from the category list on the left, and then select Spring Boot Initializr Project from the project list on the right. Then click Next to move to the next page.

The second page in the new project wizard (figure A.12) lets you set essential project information, such as the project name, version, and other information that will ultimately be used to define the project in a Maven pom.xml file.

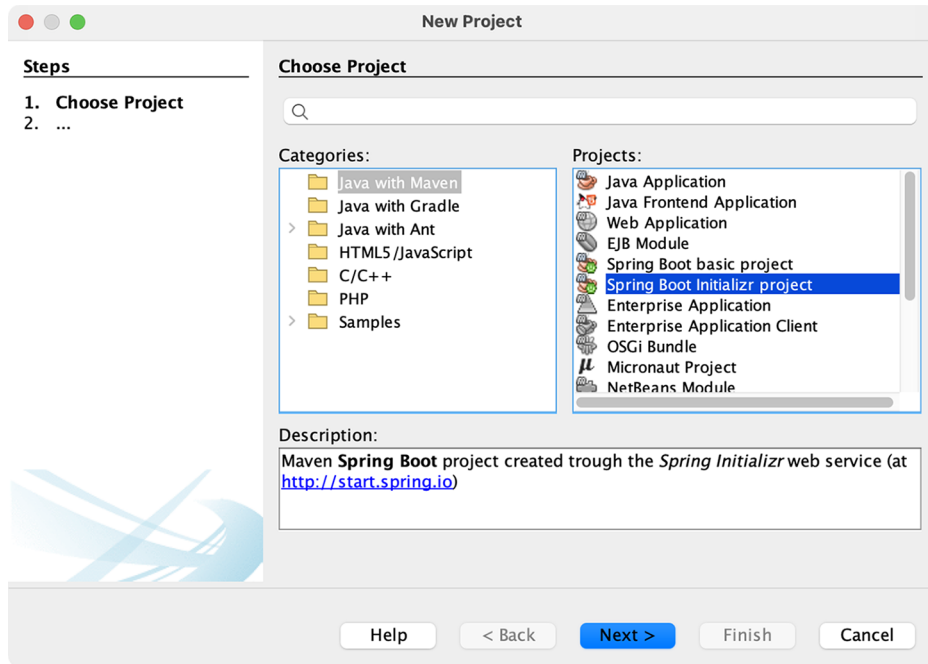


Figure A.11 Creating a new Spring Boot Initializr project

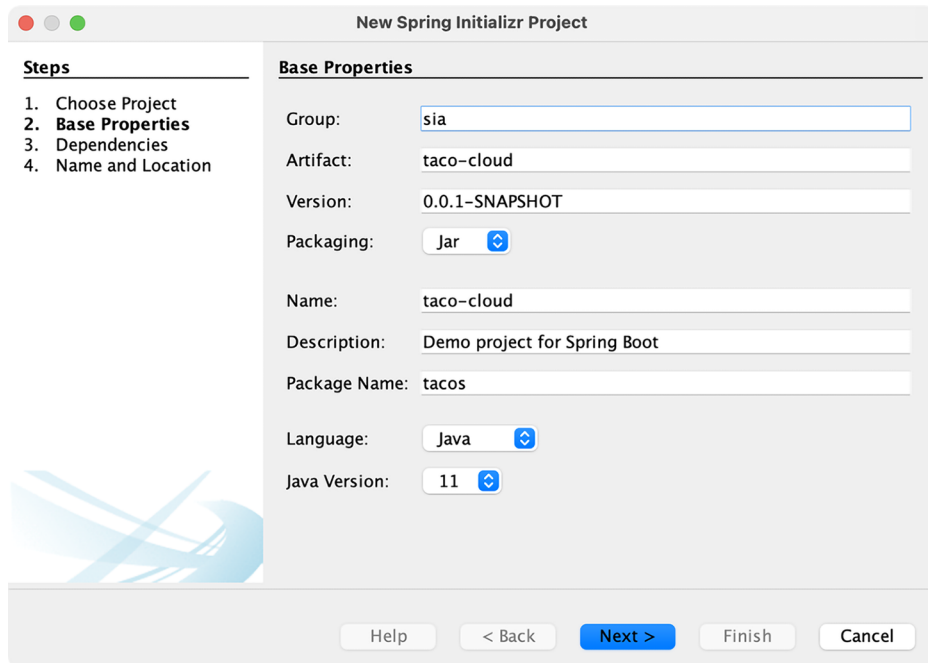


Figure A.12 Specifying essential project information

After you've specified the basic project information, click Next to navigate to the dependencies page in the new project wizard, shown in figure A.13.

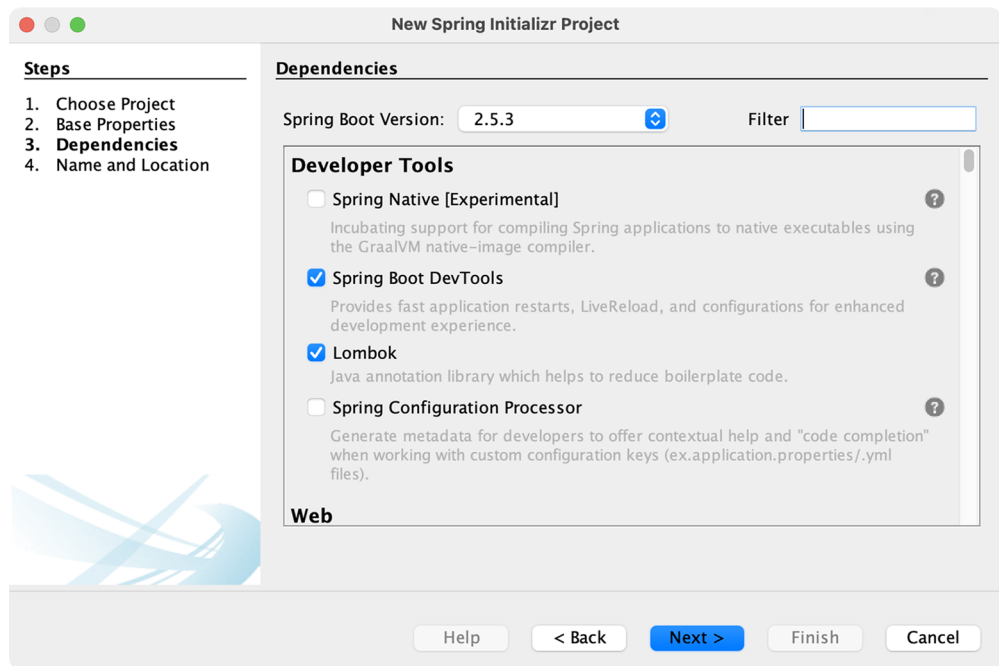


Figure A.13 Selecting project dependencies

Dependencies are all listed as check boxes in the same list, organized by category. If you have trouble finding the specific dependency you need, you can use the Filter text box at the top to limit the list of options.

You can also specify which version of Spring Boot you wish to use on this page. It will be set to the current generally available version of Spring Boot by default.

Once you've selected the dependencies for your project, click Next to navigate to the last page of the new project wizard, shown in figure A.14. This page lets you specify some final details about the project, including the project name and location on the filesystem. (The Project Folder field is read only and derived from the other two fields.) It also gives you the option to run and debug your project through the Maven Spring Boot plugin instead of through NetBeans. You may also choose to have NetBeans remove the Maven wrapper from the generated project.

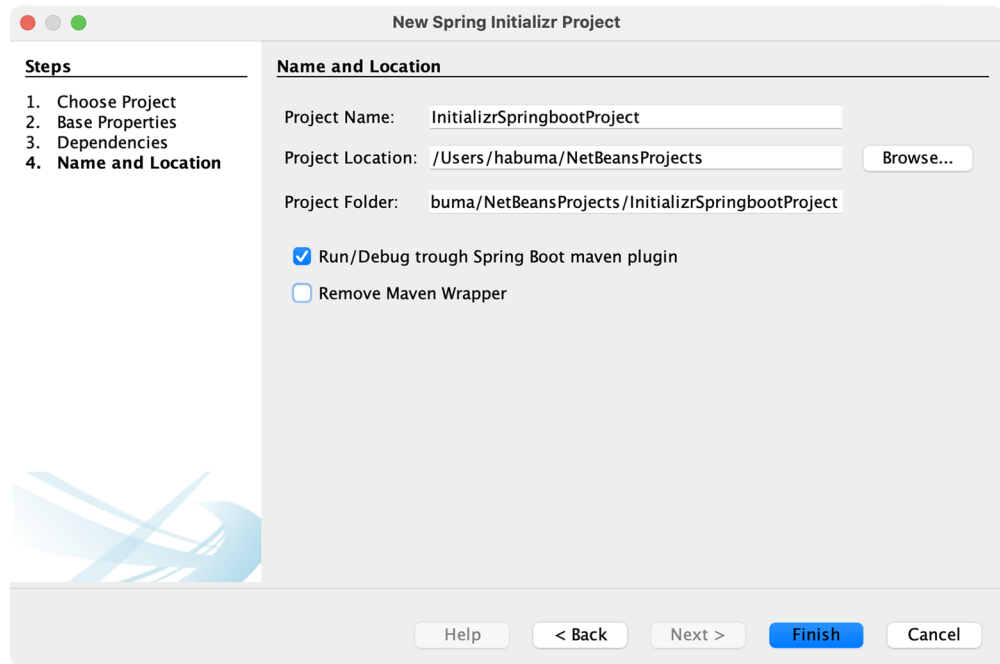


Figure A.14 Specifying the project's name and location

Once you've set the final bit of project information, click **Finish** to generate the project and have it added to your NetBeans workspace.

A.4 *Initializing a project at start.spring.io*

Although one of the IDE-based initialization options described thus far will likely suit your needs, it's possible that you may use a completely different IDE, or you might favor working with a simpler text editor. In that case, you can still take advantage of the Spring Initializr using the Initializr web-based interface.

To get started, direct your favorite web browser to <https://start.spring.io>. You should see the simple version of the Spring Initializr web user interface, shown in figure A.15.

The screenshot shows the Spring Initializr web interface in a browser window. The URL is `start.spring.io`. The interface is divided into several sections:

- Project:** Radio buttons for `Maven Project` (selected) and `Gradle Project`.
- Language:** Radio buttons for `Java` (selected), `Kotlin`, and `Groovy`.
- Spring Boot:** Radio buttons for versions `2.6.0 (SNAPSHOT)`, `2.5.4 (SNAPSHOT)`, `2.4.10 (SNAPSHOT)`, `2.6.0 (M1)`, `2.5.3` (selected), and `2.4.9`.
- Project Metadata:** Text input fields for `Group` (filled with `sia`), `Artifact` (filled with `taco-cloud`), `Name` (filled with `taco-cloud`), and `Description` (filled with `Taco Cloud Example`). A `Package name` field is filled with `sia.taco-cloud`.
- Packaging:** Radio buttons for `Jar` (selected) and `War`.
- Java:** Radio buttons for versions `16`, `11` (selected), and `8`.
- Dependencies:** A section with the text `No dependency selected` and an `ADD ...` button.

At the bottom, there are three buttons: `GENERATE` (with a keyboard shortcut `⌘ + ↵`), `EXPLORE` (with a keyboard shortcut `CTRL + SPACE`), and `SHARE...`. Social media icons for GitHub and Twitter are visible on the left side.

Figure A.15 The simple version of the Spring Initializr web interface

In the simple version of the Initializr web application, you’re asked for some very basic information, including whether you want to build with Maven or Gradle, which language you want to develop the project with, which version of Spring Boot to build against, and the group and artifact IDs of the project.

You’ll also have the option of specifying dependencies by typing search criteria in the search box. For example, as shown in figure A.16, you can type `web` to search for any dependencies where “web” is a keyword.

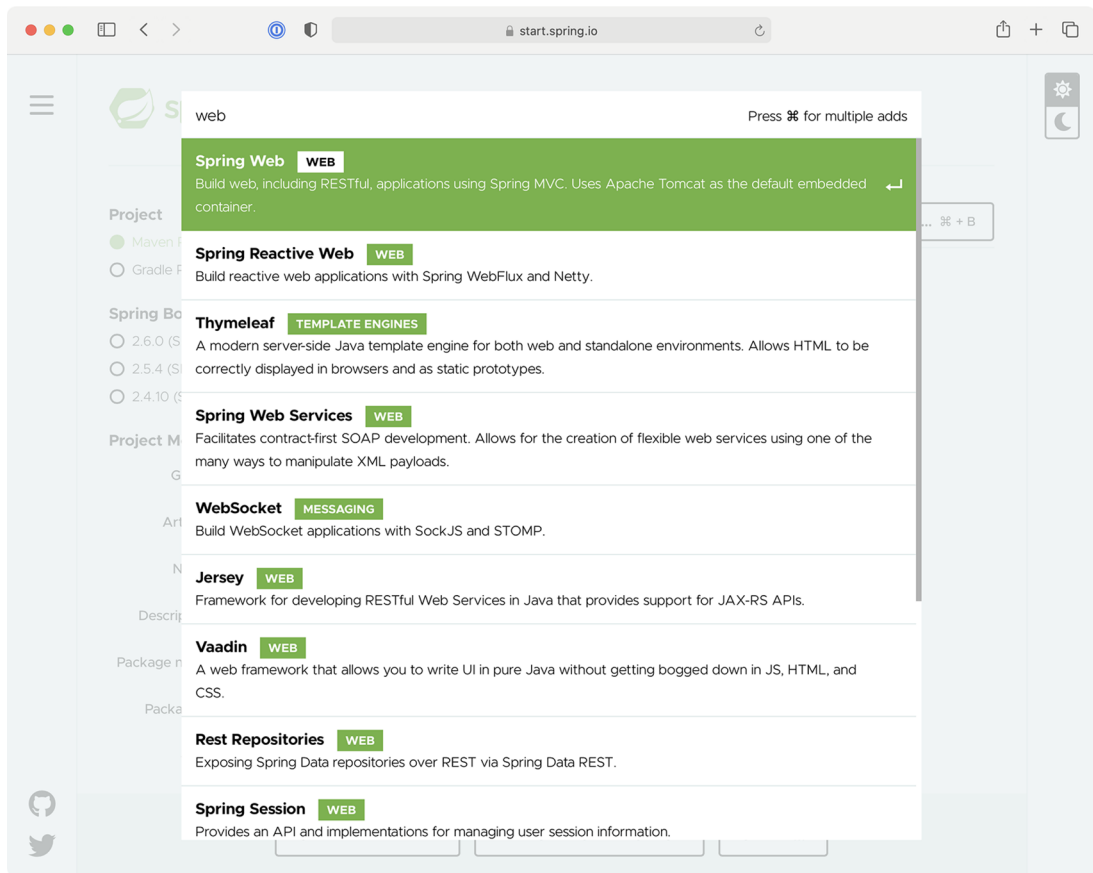


Figure A.16 Searching for dependencies

When you see the dependency you want, press Return on your keyboard to select it, and it will be added to the list of selected dependencies. The boxes beneath Selected Dependencies in figure A.17 show that the Web, Thymeleaf, DevTools, and Lombok dependencies have been selected.

If you decide you don't need a selected dependency, you can click the X to the right of the dependency entry to remove it. When you're finished, you can click Generate Project (or use the keyboard shortcut displayed on the button, which will vary by operating system) to have the Initializr generate the project and download it as a zip file. Then you can unzip the project and load it in whatever IDE or editor you choose.

The screenshot shows the Spring Initializr web application interface. The browser address bar displays `start.spring.io`. The interface is divided into several sections:

- Project:**
 - ☒ Maven Project
 - ☐ Gradle Project
- Language:**
 - ☒ Java
 - ☐ Kotlin
 - ☐ Groovy
- Spring Boot:**
 - ☐ 2.6.0 (SNAPSHOT)
 - ☐ 2.5.4 (SNAPSHOT)
 - ☐ 2.4.10 (SNAPSHOT)
 - ☒ 2.6.0 (M1)
 - ☒ 2.5.3
 - ☐ 2.4.9
- Project Metadata:**
 - Group:**
 - Artifact:**
 - Name:**
 - Description:**
 - Package name:**
 - Packaging:** ☒ Jar ☐ War
 - Java:** ☐ 16 ☒ 11 ☐ 8
- Dependencies:**
 - Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container. ☒
 - Thymeleaf** (TEMPLATE ENGINES): A modern server-side Java template engine for both web and standalone environments. Allows HTML to be correctly displayed in browsers and as static prototypes. ☒
 - Spring Boot DevTools** (DEVELOPER TOOLS): Provides fast application restarts, LiveReload, and configurations for enhanced development experience. ☒
 - Lombok** (DEVELOPER TOOLS): Java annotation library which helps to reduce boilerplate code. ☒

At the bottom, there are three buttons: **GENERATE** (with a keyboard shortcut `⌘ + ↵`), **EXPLORE** (with a keyboard shortcut `CTRL + SPACE`), and **SHARE...**

Figure A.17 Selecting dependencies

Before clicking Generate Project, you can get a sneak peak of the project by clicking Explore. This will pull up a dialog with a project explorer, much like the one shown in figure A.18.

The project's build specification (either a Maven `pom.xml` file or Gradle `build.gradle` file) will be shown first. By clicking on items in the tree on the left, you can see what other artifacts will be included in the project.

A.5 *Initializing a project from the command line*

The IDE and browser-based user interfaces for the Spring Initializr are probably the most common way that you'll bootstrap your projects. They're all just clients of a REST service offered by the Initializr application. In some special cases (e.g., in a scripted scenario), you might find it useful to consume the Initializr service directly from the command line.

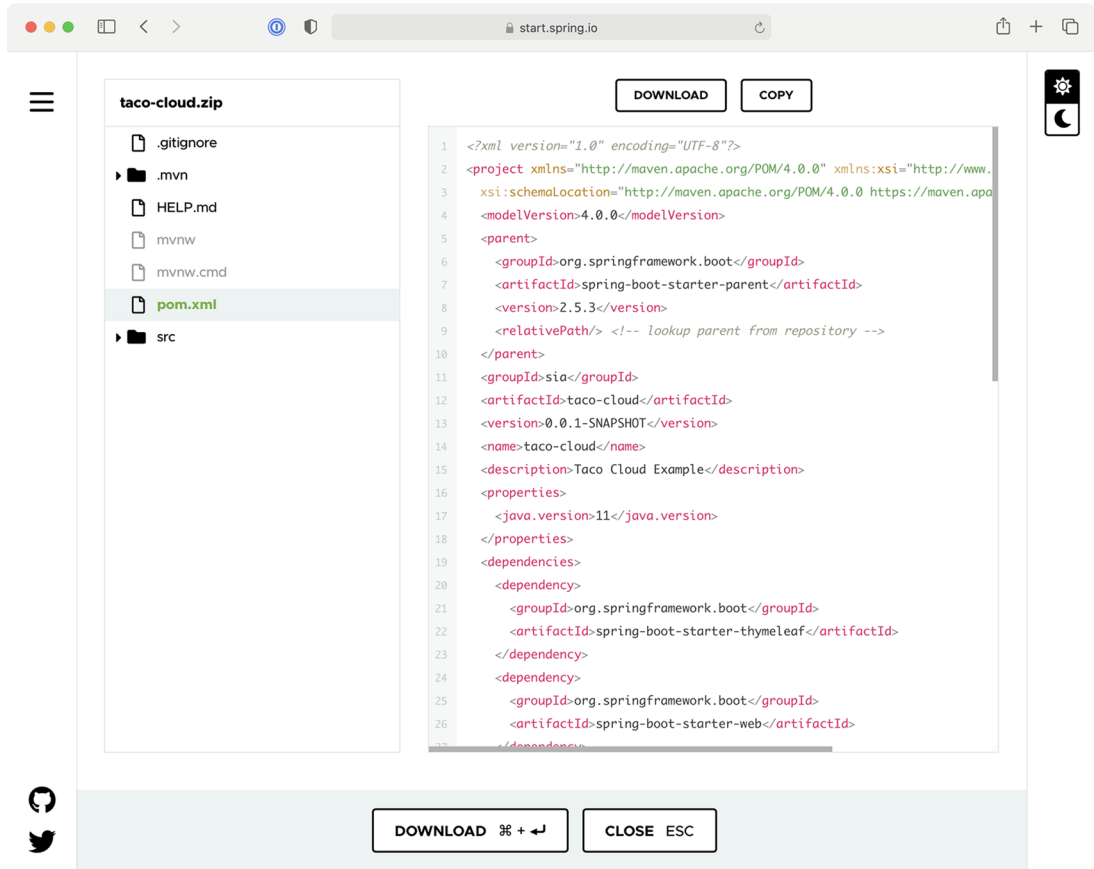


Figure A.18 The full version of the Initializr user interface

You can consume the API in the following two ways:

- Using the `curl` command (or some similar command-line REST client command)
- Using the Spring Boot command-line interface (aka, Spring Boot CLI)

Let's look at these options, starting with the `curl` command.

curl and the Initializr API

The simplest way to bootstrap a Spring project with `curl` is to consume the API like this:

```
% curl https://start.spring.io/starter.zip -o demo.zip
```

In this case, you're requesting the `/starter.zip` endpoint from the Initializr, which will generate a Spring project and download it as a zip file. The generated project will be Maven built and will have no dependencies other than the base Spring Boot

starter dependency. All project information in the project's pom.xml file will be set to default values.

If you don't specify otherwise, the name of the file will be starter.zip. But in this case, the -o option specifies that the downloaded file should be named demo.zip.

The publicly available Spring Initializr server is hosted at <https://start.spring.io>, but if you're using a custom Initializr, you'll need to adapt the given URL accordingly.

You'll probably want to specify a few more details and dependencies beyond the given defaults. Table A.1 lists all of the parameters (and their defaults) when consuming the Spring Initializr REST service.

Table A.1 Request parameters supported by the Initializr API

Parameter	Description	Default value
groupId	The project's group ID, for the sake of organization in a Maven repository	com.example
artifactId	The project's artifact ID, as it would appear in a Maven repository	demo
version	The project version	0.0.1-SNAPSHOT
name	The project name; also used to determine the name of the application's main class (with an Application suffix)	demo
description	The project description	Demo project for Spring Boot
packageName	The project's base package name	com.example.demo
dependencies	Dependencies to include in the project's build specification	The base Spring Boot starter
type	The kind of project to generate: either maven-project or gradle-project	maven-project
javaVersion	The version of Java to build with	1.8
bootVersion	The version of Spring Boot to build against	The current GA version of Spring Boot
language	The programming language to use: java, groovy, or kotlin	java
packaging	How the project should be packaged: either jar or war	jar
applicationName	The name of the application	The value of the name parameter
baseDir	The name of the base directory in the generated archive	The root directory

You can also get this list of parameters, as well as a list of available dependencies, by making a simple request to the base Initializr URL as follows:

```
% curl https://start.spring.io
```

The `dependencies` parameter is the one you'll probably find the most useful. For example, suppose that you want to create a simple web project with Spring. The following command-line use of `curl` will produce a project zip with the web starter as a dependency:

```
% curl https://start.spring.io/starter.zip \
  -d dependencies=web \
  -o demo.zip
```

As a more complex example, suppose you wanted to develop a web application that uses Spring Data JPA for data persistence. You also want to build it with Gradle, and the project should be under a directory named `my-dir` within the zip file. And let's suppose that rather than just download a zip file, you want the project unpacked into your filesystem upon download. In that case, the following command should do the trick:

```
% curl https://start.spring.io/starter.tgz \
  -d dependencies=web,data-jpa \
  -d type=gradle-project \
  -d baseDir=my-dir | tar -xzf -
```

Here, the downloaded zip file is piped to the `tar` command for unpacking.

Spring Boot command-line interface

The Spring Boot CLI is another option for initializing Spring applications. You can install the Spring Boot CLI in many ways, but probably the easiest way (and my favorite) is to use SDKMAN (<http://sdkman.io/>), as shown next:

```
% sdk install springboot
```

Once the Spring Boot CLI is installed, you can start using it to generate projects, much like with `curl`. The command you'll use is `spring init`. In fact, the simplest way to use the Spring Boot CLI to generate a project is like this:

```
% spring init
```

This will result in a bare-bones Spring Boot project being downloaded in a zip file named `demo.zip`. However, you'll probably want to specify more details and dependencies. Table A.2 lists all of the parameters available to the `spring init` command.

Table A.2 Request parameters supported by the `spring init` command

Parameter	Description	Default value
<code>group-id</code>	The project's group ID, for the sake of organization in a Maven repository	<code>com.example</code>
<code>artifact-id</code>	The project's artifact ID, as it would appear in a Maven repository	<code>demo</code>
<code>version</code>	The project version	<code>0.0.1-SNAPSHOT</code>
<code>name</code>	The project name; also used to determine the name of the application's main class (with an <code>Application</code> suffix)	<code>demo</code>
<code>description</code>	The project description	Demo project for Spring Boot
<code>package-name</code>	The project's base package name	<code>com.example.demo</code>
<code>dependencies</code>	Dependencies to include in the project's build specification	The base Spring Boot starter
<code>type</code>	The kind of project to generate: either <code>maven-project</code> or <code>gradle-project</code>	<code>maven-project</code>
<code>java-version</code>	The version of Java to build with	<code>11</code>
<code>boot-version</code>	The version of Spring Boot to build against	The current GA version of Spring Boot
<code>language</code>	The programming language to use: <code>java</code> , <code>groovy</code> , or <code>kotlin</code>	<code>java</code>
<code>packaging</code>	How the project should be packaged: either <code>jar</code> or <code>war</code>	<code>jar</code>

You can also get this list of parameters, as well as a list of available dependencies, by using the `--list` parameter as follows:

```
% spring init --list
```

Suppose you wish to create a web application that builds against Java 1.7. The following command uses the `--dependencies` and `--java` parameters to make those choices:

```
% spring init --dependencies=web --java-version=1.7
```

Or suppose you want to create a web application with Spring Data JPA for persistence, and you'd like to use Gradle to perform the build instead of Maven. You'd use the following command:

```
% spring init --dependencies=web,jpa --type=gradle-project
```

You may also notice that many of the `spring init` parameters are the same as or similar to the parameters for the `curl` option. That said, the `spring init` command doesn't support all of the same parameters as the `curl` option (e.g., `baseDir`), and the parameters are hyphen-delimited instead of camelCase (e.g., `package-name` versus `packageName`).

A.6 *Building and running projects*

No matter how you initialize your project, you can always run the application from the command line with the `java -jar` command as follows:

```
% java -jar demo.jar
```

This will even work if you decide to create a WAR file distribution instead of a JAR file, as shown here:

```
% java -jar demo.war
```

You can also take advantage of the Spring Boot Maven and Gradle plugins to run your application. For example, if your project is built with Maven, you can run it like this:

```
% mvn spring-boot:run
```

If, on the other hand, you've chosen to build your project with Gradle, you can run your project like this:

```
% gradle bootRun
```

In either case, whether using Maven or Gradle, the build tool will first build your project (if it hasn't already been built) and run it.

A

- AbstractMessageRouter bean 256
- access() method 126
- action attribute 41
- activating profiles 158–159
- ActiveMQ in Action* (Snyder, Bosanac, and Davies) 211
- Actuator 388–391
 - configuring base path 389–390
 - consuming endpoints 391–408
 - application information 392–395
 - runtime metrics 405–408
 - viewing application activity 403–405
 - viewing configuration details 395–403
 - customizing 408–419
 - endpoints 417–419
 - health indicators 414–415
 - registering metrics 415–417
 - the /info endpoint 408–413
 - enabling and disabling endpoints 390–391
 - securing 420–421
- Actuator MBeans
 - creating 437–440
 - working with 435–437
- addIngredient() method 207
- addIngredientsToModel() method 70
- addNote() method 419
- addScript() method 141
- addScripts() method 141
- addTaco() method 34, 348
- addViewController() method 56
- addViewControllers() method 56, 129
- administering Spring
 - Admin server 427–431
 - environment properties 429–431
 - general application health and information 428
 - key metrics 428–429
 - logging levels 431
 - securing 431–433
 - using Spring Boot Admin 424–427
 - creating Admin server 424–426
 - registering Admin clients 426–427
 - Admin server 427–431
 - creating 424–426
 - environment properties 429–431
 - general application health and information 428
 - key metrics 428–429
 - logging levels 431
 - securing 431–433
 - authenticating with Actuator 433
 - enabling login 432–433
- aggregate root service, OrderRepository 347–353
- Alert object 378
- allIngredients() method 401
- all operation 306
- AMQP 211, 226–236
 - adding RabbitMQ to Spring 227–228
 - receiving message from RabbitMQ 232–236
 - handling RabbitMQ messages with listeners 235–236
 - receiving messages with RabbitTemplate 232–235
 - sending messages with RabbitTemplate 228–232
 - configuring message converter 231
 - setting message properties 231–232
- and() method 129
- any() operation 306
- ApiProperties object 273

APIs, reactive

- consuming REST APIs reactively 325–332
 - deleting resources 329
 - exchanging requests 331–332
 - GETting resources 326–328
 - handling errors 329–331
 - sending resources 328–329
 - functional request handlers 316–320
 - securing reactive web APIs 333–336
 - configuring reactive user details service 335–336
 - configuring reactive web security 333–335
 - Spring WebFlux 309–316
 - overview 310–311
 - writing reactive controllers 312–316
 - testing reactive controllers 320–325
 - GET requests 320–323
 - POST requests 323–324
 - with live server 324–325
- ApplicationArguments parameter 84
- applicationConfig property 398
- ApplicationRunner bean 374
- applications
- configuration details 395–403
 - autoconfiguration 396–397
 - bean wiring report 395–396
 - HTTP request mappings 400–401
 - inspecting environment and configuration properties 397–400
 - managing logging levels 401–403
 - information 392–395
 - asking for 392
 - inspecting health 393–395
 - Spring
 - building and running projects 477
 - initializing project from command line 472–477
 - IntelliJ IDEA 463–464
 - NetBeans 465–469
 - Spring Tool Suite 459–462
 - start.spring.io 469–472
 - viewing activity 403–405
 - monitoring threads 404–405
 - tracing HTTP activity 403–404
- args parameter 85
- array type 341
- <artifactId> property 447
- assertOrder() method 352
- asynchronous messaging
- with JMS 211–226
 - JmsTemplate 214–222
 - receiving messages 222–226
 - setting up 211–214
 - with Kafka 236–242
 - sending messages with KafkaTemplate 238–240
 - setting up Spring for 237–238
 - writing Kafka listeners 241–242

- with RabbitMQ and AMQP 226–236
 - adding RabbitMQ to Spring 227–228
 - receiving message from RabbitMQ 232–236
 - sending messages with RabbitTemplate 228–232
- authentication 116–124
 - customizing user authentication 119–124
 - in-memory user details service 118–119
- authorization code grant 190
- authorization servers 192–201
- authorizationServerSecurityFilterChain() bean
 - method 195
- autoconfiguration 141–148
 - data source 143–144
 - embedded server 145
 - logging 146–147
 - overview 396–397
 - Spring environment abstraction 142–143
 - using special property values 148
- automatic features
 - automatic application restart 24
 - automatic browser refresh and template cache
 - disable 24–25

B

- base path, Actuator 389–390
- base URI 327
- @Bean() method 125, 141, 252, 317, 454
- @Bean-annotated() method 84, 141
- beans
 - conditionally creating with profiles 159–160
 - wiring report 395–396
- Between operation 91
- Bosanac, Dejan 211
- browser refresh, automatic 24–25
- BSON (Binary JSON) format 106
- buffer() operation 302
- buffering data 302–305
- build() method 335
- Builder object 409
- build/libs directory 445

C

- caching templates 59
- Carnell, John 426
- cascade attribute 89
- Cassandra
 - reactively persisting data in 361–368
 - creating reactive Cassandra repositories 365–366
 - domain classes for 362–365
 - testing reactive Cassandra repositories 366–368
 - repositories 95–105
 - data modeling 98–99

- Cassandra, repositories (*continued*)
 - enabling Spring Data Cassandra 95–98
 - mapping domain types for Cassandra persistence 99–105
 - writing 105
 - Cassandra CQL (Cassandra Query Language) 96
 - ccCVV property 51
 - ccExpiration property 51
 - ccNumber property 51
 - cf command-line tool 446
 - channel adapters 263–265
 - class attribute 54
 - clients
 - creating RSocket 372–382
 - fire-and-forget messages 378–379
 - request-response 372–375
 - request-stream messaging 376–377
 - developing 204–209
 - sending messages bidirectionally 379–382
 - Cloud Native Spring in Action* (Vitale) 28
 - cloud profile 158
 - collection attribute 108
 - collections, creation operations from 288–290
 - collectList() operation 304
 - collectMap() operation 304
 - combination operations 291–295
 - merging 291–294
 - selecting first reactive type to publish 294–295
 - command line 472–477
 - curl and Initializr API 473–475
 - Spring Boot command-line interface 475–477
 - CommandLineRunner 83–85
 - CommandLineRunner bean 84, 159, 167, 194
 - @ConditionalOnMissingBean annotation 397
 - configuration() method 333
 - @Configuration-annotated class 160
 - @Configuration annotation 5
 - @Configuration class 317
 - configuration properties
 - autoconfiguration 141–148
 - data source 143–144
 - embedded server 145
 - logging 146–147
 - Spring environment abstraction 142–143
 - using special property values 148
 - creating 148–155
 - declaring metadata 153–155
 - defining holders 151–153
 - inspecting 397–400
 - with profiles 155–160
 - activating 158–159
 - conditionally creating beans with 159–160
 - profile-specific properties 156–157
 - @ConfigurationProperties-annotated class 234
 - @ConfigurationProperties annotation 148
 - ConfigurationPropertiesAutoConfiguration bean 397
 - configure() method 334, 420, 456
 - configuredLevel property 402
 - consumes attribute 170
 - container images 446–455
 - deploying to Kubernetes 449–451
 - enabling graceful shutdown 451–452
 - liveness and readiness 452–455
 - configuring probes in deployment 453–454
 - enabling probes 452–453
 - managing 454–455
 - contexts element 396
 - contribute() method 409
 - controllers
 - creating class for 34–38
 - testing 20–21
 - view controllers 54–57
 - writing reactive 312–316
 - handling input reactively 315–316
 - returning single values 314
 - RxJava types 314–315
 - writing RESTful controllers 164–174
 - deleting data from server 173–174
 - retrieving data from server 164–170
 - sending data to server 170–171
 - updating data on server 171–173
 - convert() method 43
 - convertAndSend() method 215
 - Converter interface 43
 - converters, message
 - configuring with RabbitTemplate 231
 - JmsTemplate
 - before sending 218–219
 - configuring 219–220
 - core Spring Framework 26
 - count() method 409
 - createdAt property 101
 - createdDate property 179
 - create keyspace command 96, 361
 - creation operations 287–291
 - from collections 288–290
 - from objects 287–288
 - generating Flux data 290–291
 - @CrossOrigin annotation 166
 - cross-site request forgery 133–134
 - CrudRepository interface 89, 343
 - curl command 402, 473
 - curl command-line client 391
 - curl option 477
- ## D
-
- data
 - buffering on reactive stream 302–305
 - filtering from reactive types 295–299

- data (*continued*)
 - mapping reactive 299–302
 - nonrelational data
 - Cassandra repositories 95–105
 - MongoDB repositories 106–111
 - persisting data with Spring Data JPA 85–93
 - adding to project 85–86
 - annotating domain as entities 86–89
 - customizing repositories 90–93
 - declaring JPA repositories 89–90
 - persisting document data reactively 353–360
 - domain document types 354–356
 - reactive MongoDB repositories 356–357
 - testing reactive MongoDB repositories 357–360
 - reading and writing data with JDBC 62–78
 - adapting domain for persistence 64–65
 - inserting data 73–78
 - JdbcTemplate 65–70
 - schema and preloading data 70–73
 - RESTful controllers
 - deleting data from server 173–174
 - retrieving data from server 164–170
 - sending data to server 170–171
 - updating data on server 171–173
 - Spring Data JDBC 78–85
 - adding to build 78–79
 - annotating domain for persistence 81–83
 - preloading data with CommandLineRunner 83–85
 - repository interfaces 79–80
 - @Data annotation 32, 87
 - data-backed services 174–180
 - adjusting resource paths and relation names 177–179
 - paging and sorting 179–180
 - @DataMongoTest annotation 359
 - data parameter 245
 - @DataR2dbcTest annotation 345
 - DataSource bean 141
 - Davies, Rob 211
 - DDD (domain-driven design) 71
 - defaultRequestChannel attribute 245
 - delayElements() operation 292
 - delaySubscription() operation 292
 - delete() method 184, 329
 - deleteAllOrders() method 135
 - deleteById() method 174
 - deleteNote() method 419
 - deleteOrder() method 174
 - DELETE request 173, 201, 400
 - deleting resources 184, 329
 - deliveryName property 82
 - deliveryZip property 90
 - dependencies 286
 - <dependencies> element 14
 - dependencies parameter 476
 - dependencies parameter 475
 - dependencies property 396
 - deploying Spring
 - container images 446–455
 - deploying to Kubernetes 449–451
 - enabling graceful shutdown 451–452
 - liveness and readiness 452–455
 - executable JAR files 445–446
 - options for 444–445
 - WAR files 455–457
 - description property 154
 - DesignTacoController class 35
 - Destination bean 217
 - Destination object 217
 - dev profile 159
 - DI (dependency injection) 4
 - diagramming reactive flows 285–286
 - disabling endpoints 390–391
 - display, web applications 30–41
 - creating controller class 34–38
 - designing view 38–41
 - establishing domain 31–34
 - distinct() operation 298
 - <div> element 39
 - docker build command 446
 - @Document annotation 108, 354
 - document data 353–360
 - domain document types 354–356
 - reactive MongoDB repositories 356–357
 - testing reactive MongoDB repositories 357–360
 - domain-driven design (DDD) 71
 - Domain-Driven Design (Evans) 31, 71
 - domains
 - adapting for persistence 64–65
 - annotating as entities 86–89
 - annotating for persistence 81–83
 - entities for R2DBC 339–343
 - establishing 31–34
 - for Cassandra persistence 99–105, 362–365
 - mapping MongoDB repositories to documents 107–110
 - doTransform() method 272
 - DSL configuration 249–251
-
- E**
- effectiveLevel property 402
 - email integration flow 267–274
 - EmailOrder class 272
 - EmailOrder object 272
 - EmailProperties class 268
 - EmailToOrderTransformer class 270
 - enabling endpoints 390–391
 - EndpointRequest.toAnyEndpoint() method 421

- endpoints 391–408
 - application activity 403–405
 - monitoring threads 404–405
 - tracing HTTP activity 403–404
 - application information 392–395
 - asking for 392
 - inspecting application health 393–395
 - configuration details 395–403
 - autoconfiguration 396–397
 - bean wiring report 395–396
 - HTTP request mappings 400–401
 - inspecting environment and configuration properties 397–400
 - managing logging levels 401–403
 - customizing 417–419
 - enabling and disabling 390–391
 - modules 265–267
 - runtime metrics 405–408
 - the /info endpoint 408–413
 - build information 410–411
 - Git commit information 411–413
 - InfoContributor 408–410
- Enterprise Integration Patterns* (Hohpe and Woolf) 244
- environment properties 397–400
- Equals operation 91
- errors 54, 329–331
- Evans, Eric 31, 71
- event looping 309
- exception tag 408
- exchangeToFlux() method 332
- exchangeToMono() method 331
- exchanging requests 331–332
- expectBodyList() method 322
- Expert One-on-One J2EE Design and Development* (Johnson) 3

F

- failOnNoGitDirectory property 411
- fields property 54
- FIFO (first in, first out) 252
- filename parameter 245
- Files type 250
- FileWriterGateway interface 251
- FileWritingMessageHandler bean 249
- filter() method 254
- filter() operation 298
- @Filter annotation 254
- filterChain() method 125
- filtering data 295–299
- filters 253–254
- final property 32
- findAll() method 70, 166, 207, 312, 347
- findById() method 68, 169, 343

- findBySlug() method 345
- findByUsername() method 121
- findByUserOrderByPlacedAtDesc() method 149
- fire-and-forget messages 378–379
- firstWithSignal() operation 294
- flatMap() operation 299, 320, 346
- Flux data 290–291
- force attribute 87
- forgery, cross-site request 133–134
- <form> element 41, 134
- forms
 - processing submissions 41–49
 - validating form input 49–54
 - declaring validation rules 50–52
 - displaying validation errors 54
 - performing validation at form binding 52–54
- frameworkless framework 25
- from() method 264
- fromArray() method 289, 382
- fromIterable() method 289
- Functional Programming in Java* (Saumont) 281
- functional request handlers 316–320

G

- Gamov, Viktor 237
- gateways 262–263
- GeneratedKeyHolder type 74
- GenericHandler interface 273
- GenericSelector interface 254
- GenericTransformer interface 255
- GET() method 317
- get() method 169
- getAuthorities() method 121
- getContent() method 312
- getForEntity() method 183
- getForObject() method 183, 326
- getImapUrl() method 270
- @GetMapping-annotated() method 173
- @GetMapping annotation 42, 166
- getMessageConverter() method 229
- GET requests 18, 35, 37–38, 123, 164, 319–323, 389, 429, 454
- getTacoCount() method 439
- GETting resources 182–183, 326–328
 - making requests with base URI 327
 - timing out on long-running requests 328
- Git commit information 411–413
- graceful shutdown, enabling 451–452
- GratuityIn class 380
- GratuityOut class 380
- gratuity property 380
- Grokking Functional Programming* (Plachta) 281

H

H2 Console 25
 handle() method 241, 273
 handleGreeting() method 373
 .hasAuthority() method 202
 hasErrors() method 53
 hasRole() method 126
 @Header annotation 245
 HealthIndicator interface 414
 health indicators 414–415
 helloRouterFunction() method 318
 Hohpe, Gregor 244
 holders, configuration property 151–153
 home() method 18
 homepages 19–20
 HTTP
 request mappings 400–401
 tracing activity 403–404
 HttpSecurity object 125, 334
 HttpStatus object 331

I

@Id annotation 82, 108
 id field 64
 id parameter 169
 @Id property 109
 id property 76, 100, 184
 if statement 310
 if/then blocks 49
 -imageName parameter 448
 tag 19
 @ImportResource annotation 247
 import statement 25
 @InboundChannelAdapter annotation 264
 increment() method 439
 info.contact.email property 392
 info.contact.phone property 392
 InfoContributor 408–410
 /info endpoint 408–413
 build information 410–411
 Git commit information 411–413
 InfoContributor 408–410
 Ingredient class 31, 64, 99, 339
 Ingredient data 66
 Ingredient entity 177
 ingredientId parameter 182
 Ingredient object 64, 182, 326, 340
 IngredientRepository.findById() method 70
 IngredientRepository interface 66, 111, 177, 356
 ingredientsController bean 396
 IngredientUDT class 102, 363
 initializing applications 6–17
 examining project structure 11–17

 bootstrapping application 15–16
 build specification 12–15
 testing application 16–17
 with Spring Tool Suite 7–11
 in-memory user details service 118–119
 <input> element 39
 inputChannel attribute 253
 insert() method 111
 insert query 75
 integration flow 244–251
 configuring in Java 247–249
 using DSL configuration 249–251
 with XML 246–247
 IntelliJ IDEA 463–464
 interfaces, MongoDB repositories 111
 interval() method 291
 int-file namespace 247
 InventoryService bean 5
 is* method 121

J

JAR files, building executable 445–446
 Java, configuring integration flow in 247–249
 java -jar command 477
 Java Message Service. *See* JMS
 -java parameter 476
 javax.persistence package 87
 JDBC (Java Database Connectivity) 61
 reading and writing data with 62–78
 adapting domain for persistence 64–65
 inserting data 73–78
 JdbcTemplate 65–70
 schema and preloading data 70–73
 Spring Data JDBC 78–85
 adding to build 78–79
 annotating domain for persistence 81–83
 preloading data with CommandLineRunner 83–85
 repository interfaces 79–80
 JdbcIngredientRepository bean 67
 JdbcTemplate 65–70
 defining JDBC repositories 66–68
 inserting rows 68–70
 JMS (Java Message Service) 211–226
 JmsTemplate 214–222
 configuring message converter 219–220
 converting messages before sending 218–219
 postprocessing messages 220–222
 receiving messages 222–226
 declaring message listeners 224–226
 with JmsTemplate 222–224
 setting up 211–214
 @JmsListener annotation 225
 JmsTemplate() method 228

JmsTemplatesConvertAndSend() method 218
 JMX
 Actuator MBeans
 creating 437–440
 working with 435–437
 sending notifications 440–442
 JNDI (Java Naming and Directory Interface)
 144
 Johnson, Rod 3
 json() method 322
 just() method 287
 JWK (JSON Web Key) 196
 JWT (JSON Web Token) 191

K

Kafka 236–242
 sending messages with KafkaTemplate 238–240
 setting up Spring for 237–238
 writing Kafka listeners 241–242
Kafka in Action (Scott, Gamov, and Klein) 237
 @KafkaListener annotation 241
 Klein, Dave 237
 kubectl command-line tool 450
 kubectl port-forward command 451
 Kubernetes, deploying to 449–451
Kubernetes in Action, 2nd Edition (Lukša) 449

L

ListItem object 259
 listItemSplitter() method 259
 ListBodySpec object 323
 listeners
 declaring for JMS 224–226
 handling RabbitMQ messages with 235–236
 writing Kafka 241–242
 --list parameter 476
 List<Taco> property 348
 liveness and readiness 452–455
 configuring probes in deployment 453–454
 enabling probes 452–453
 managing 454–455
 live servers 324–325
 loadUserByUsername() method 118
 log() operation 303
 Logger object 45
 loggers element 402
 logging 401–403
 logging.file.name property 147
 logging.file.path property 147
 logging.level.tacos property 156
 logic operations 305–307
 login pages 128–130
 long parameter 233

long-running requests 328
 Lukša, Marko 449

M

main() method 16
 management.endpoint.health.show-details
 property 393
 management.endpoints.web.exposure.include
 property 390
 management.endpoint.web.base-path
 property 389
 management.info.git.mode property 412
 @ManyToOne annotation 137
 map() function 373
 map() method 349
 map() operation 284, 336, 350
 mapping
 HTTP request 400–401
 reactive data 299–302
 mapRowToIngredient() method 68
 matches() method 117
 memory, in-memory user details service 118–119
 mergeWith() method 292
 mergeWith() operation 291
 merging reactive types 291–294
 message attribute 52
 message channels 252–253
 MessageHandler bean 261
 MessageHandler interface 262
 @MessageMapping annotation 373
 Message object 272
 MessageProperties object 232
 messaging
 fire-and-forget messages 378–379
 request-stream messaging 376–377
 sending bidirectionally 379–382
 with JMS 211–226
 JmsTemplate 214–222
 receiving messages 222–226
 setting up 211–214
 with Kafka 236–242
 sending messages with KafkaTemplate 238–240
 setting up Spring for 237–238
 writing Kafka listeners 241–242
 with RabbitMQ and AMQP 226–236
 adding RabbitMQ to Spring 227–228
 receiving message from RabbitMQ 232–236
 sending messages with RabbitTemplate 228–232
 metadata, configuration property 153–155
 method-level security 134–136
 method tag 408
 metrics
 registering 415–417
 runtime 405–408

microservices 28
 MockMvc object 21
 Model object 37
 MongoDB
 persisting document data reactively with 353–360
 domain document types 354–356
 reactive MongoDB repositories 356–357
 testing reactive MongoDB repositories 357–360
 repositories 106–111
 enabling Spring Data MongoDB 106–107
 mapping domain types to documents 107–110
 writing interfaces 111
 MongoTemplate bean 397
 monitoring Spring
 Actuator MBeans
 creating 437–440
 working with 435–437
 sending notifications 440–442
 Mono.just() method 349
 Mono objects 284
 Mono<String> parameter 373
 Mono type 370

N

name property 39
 NetBeans 465–469
 nonrelational data
 Cassandra repositories 95–105
 Cassandra data modeling 98–99
 enabling Spring Data Cassandra 95–98
 mapping domain types for Cassandra
 persistence 99–105
 writing 105
 MongoDB repositories 106–111
 enabling Spring Data MongoDB 106–107
 mapping domain types to documents 107–110
 writing interfaces 111
 @NotBlank annotation 51
 notes() method 418
 NotesEndpoint class 418
 Notification object 440
 NotificationPublisherAware interface 440

O

OAuth 2 187–192
 oauth2ResourceServer() method 202
 objects, creation operations from 287–288
 OIDC (OpenID Connect) 131
 onAfterCreate() method 416, 439
 onNext() method 283

-o option 474
 order() method 43
 @Order annotation 195
 OrderController class 47
 orderForm() method 45
 OrderMessagingService interface 216
 OrderProps bean 151
 OrderProps class 151
 OrderRepository aggregate root service 347–353
 OrderRepository interface 73, 111
 ordersForUser() controller method 149
 OrderSplitter bean 258
 org.springframework.data.annotation package 87
 origin field 399
 origins attribute 166

P

<p> element 38
 <packaging> element 456
 Page object 312
 page parameter 179
 PageRequest object 149
 pageSize property 150
 paging 179–180
 <parent> element 14
 PasswordEncoder bean 117
 PATCH command 171
 patchOrder() method 172
 path attribute 170
 pathMatchers() method 335
 @PathVariable annotation 374
 @Pattern annotation 52
 permitAll() method 126
 persisting data
 adapting domain for 64–65
 mapping domain types for Cassandra 99–105
 reactively
 document data with MongoDB 353–360
 in Cassandra 361–368
 R2DBC (Reactive Relational Database
 Connectivity) 338–353
 with Spring Data JPA 85–93
 adding to project 85–86
 annotating domain as entities 86–89
 customizing repositories 90–93
 declaring JPA repositories 89–90
 placedAt property 149
 Plachta, Michał 281
 Player object 300
 pollRate property 270
 post() method 328
 @PostAuthorize annotation 136
 postForObject() method 184
 POSTing resource data 184–185

- postprocessing messages 220–222
- postProcessMessage() method 232
- POST requests 41, 133, 167, 201, 323–324, 399
- postTaco() method 170, 319
- @PreAuthorize annotation 135
- prefix attribute 268
- preloading data 70–73
- private helper method 208
- private method 272
- processOrder() method 47, 78, 137
- Processor interface 283
- processPayload() method 273
- processRegistration() method 124
- processTaco() method 42
- prod profile 157
- produces attribute 166
- ProductService bean 5
- @Profile annotation 159
- profiles 155–160
 - activating 158–159
 - conditionally creating beans with 159–160
 - profile-specific properties 156–157
- project structure 11–17
 - bootstrapping application 15–16
 - build specification 12–15
 - testing application 16–17
- property field 399
- propertySources field 398
- provider property 205
- publish() method 455
- Publisher interface 282
- publishing, reactive types and 294–295
- pull model 222
- push model 222
- put() method 183
- PUT command 171
- putOrder() method 172
- PUTting resources 183–184

Q

- qa profile 159
- query() method 68
- QueueChannel bean 253

R

- R2DBC (Reactive Relational Database Connectivity) 338–353
 - domain entities for 339–343
 - OrderRepository aggregate root service 347–353
 - reactive repositories 343–345
 - testing repositories 345–347
- RabbitMQ 226–236
 - adding to Spring 227–228

- receiving message from 232–236
 - messages with listeners 235–236
 - with RabbitTemplate 232–235
 - sending messages with RabbitTemplate 228–232
 - configuring message converter 231
 - setting message properties 231–232
- RabbitMQ in Action* (Videla and Williams) 227
- RabbitMQ in Depth* (Roy) 227
- RabbitTemplate bean 227
- RabbitTemplate.receive() method 232
- range() method 290
- reactive APIs
 - consuming REST APIs reactively 325–332
 - deleting resources 329
 - exchanging requests 331–332
 - GETting resources 326–328
 - handling errors 329–331
 - sending resources 328–329
 - functional request handlers 316–320
 - securing reactive web APIs 333–336
 - configuring reactive user details service 335–336
 - configuring reactive web security 333–335
- Spring WebFlux 309–316
 - overview 310–311
 - writing reactive controllers 312–316
- testing reactive controllers 320–325
 - GET requests 320–323
 - POST requests 323–324
 - with live server 324–325
- reactively persisting data
 - document data with MongoDB 353–360
 - domain document types 354–356
 - reactive MongoDB repositories 356–357
 - testing reactive MongoDB repositories 357–360
- in Cassandra 361–368
 - creating reactive Cassandra repositories 365–366
 - domain classes for 362–365
 - testing reactive Cassandra repositories 366–368
- R2DBC 338–353
 - domain entities for 339–343
 - OrderRepository aggregate root service 347–353
 - reactive repositories 343–345
 - testing repositories 345–347
- reactive operations 287–307
 - combining reactive types 291–295
 - merging 291–294
 - selecting first reactive type to publish 294–295
 - creating reactive types 287–291
 - from collections 288–290
 - from objects 287–288
 - generating Flux data 290–291

- reactive operations (*continued*)
 - logic operations on reactive types 305–307
 - transforming reactive streams 295–305
 - buffering data on reactive stream 302–305
 - filtering data from reactive types 295–299
 - mapping reactive data 299–302
- reactive programming 280–283
- Reactive Relational Database Connectivity. *See* R2DBC
- Reactive Streams 281–283
- Reactor
 - adding dependencies 286
 - diagramming reactive flows 285–286
 - reactive operations 287–307
 - combining reactive types 291–295
 - creating reactive types 287–291
 - logic operations on reactive types 305–307
 - transforming reactive streams 295–305
 - reactive programming 280–283
- receive() method 222
- receiveAndConvert() method 223
- receiving messages
 - from JMS 222–226
 - declaring message listeners 224–226
 - with JmsTemplate 222–224
 - from RabbitMQ 232–236
 - handling RabbitMQ messages with listeners 235–236
 - with RabbitTemplate 232–235
- recents() method 319
- recentTacos() method 166, 314
- RegisteredClientRepository interface 195
- registerForm() method 123
- RegistrationController class 122
- RegistrationForm object 124
- relation names 177–179
- replicas property 450
- repositories
 - customizing 90–93
 - declaring JPA 89–90
 - defining JDBC 66–68
 - interfaces with Spring Data JDBC 79–80
 - reactive
 - Cassandra 365–368
 - MongoDB 343–347
- Repository interface 79
- @RequestBody annotation 170
- @RequestMapping annotation 36, 166
- request-response servers 372–375
- requests
 - exchanging 331–332
 - GET requests 320–323
 - GETting resources
 - making requests with base URI 327
 - timing out on long-running requests 328
 - HTTP request mappings 400–401
 - POST requests 323–324
 - request-stream messaging 376–377
- @RequestScope annotation 208
- ResourceDatabasePopulator objects 343
- resource paths 177–179
- resource property 396
- resource servers 201–204
- ResponseEntity object 166
- REST
 - consuming 180–185
 - DELETEing resources 184
 - GETting resources 182–183
 - POSTing resource data 184–185
 - PUTting resources 183–184
 - consuming REST APIs reactively 325–332
 - deleting resources 329
 - exchanging requests 331–332
 - GETting resources 326–328
 - handling errors 329–331
 - sending resources 328–329
 - enabling data-backed services 174–180
 - adjusting resource paths and relation names 177–179
 - paging and sorting 179–180
 - securing
 - creating authorization server 192–201
 - developing client 204–209
 - OAuth 2 187–192
 - securing API with resource server 201–204
 - writing RESTful controllers 164–174
 - deleting data from server 173–174
 - retrieving data from server 164–170
 - sending data to server 170–171
 - updating data on server 171–173
- restart, automatic application 24
- @RestController-annotated classes 175
- @RestController annotation 165
- @RestResource annotation 179
- RestTemplate methods 184
- retrieve() method 326
- rootProject.name property 445
- route() method 317
- routerFunction() method 319
- routers 256–257
- rows, JdbcTemplate 68–70
- Roy, Gavin 227
- RSocket
 - creating server and client 372–382
 - fire-and-forget messages 378–379
 - request-response 372–375
 - request-stream messaging 376–377
 - sending messages bidirectionally 379–382
 - overview 370–372
 - transporting over WebSocket 382–383

run() method 16, 83
 RxJava types 314–315

S

Sanchez, Illary Huaylupo 426
 Saumont, Pierre-Yves 281
 save() method 68, 171, 315, 346
 saveAll() method 315, 349
 saveIngredientRefs() method 76
 saveTaco() method 76
 schema, with JDBC 70–73
 Scott, Dylan 237
 security
 Actuator 420–421
 reactive web APIs 333–336
 configuring reactive user details service 335–336
 configuring reactive web security 333–335
 REST
 creating authorization server 192–201
 developing client 204–209
 OAuth 2 187–192
 securing API with resource server 201–204
 Spring
 applying method-level security 134–136
 configuring authentication 116–124
 enabling Spring Security 114–116
 knowing user 136–139
 web requests 125–134
 creating custom login page 128–130
 enabling third-party authentication 131–133
 overview 125–128
 preventing cross-site request forgery 133–134
 SecurityConfig class 201
 SecurityFilterChain bean 125, 204
 securityWebFilterChain() method 334
 send() method 215
 sending messages
 bidirectionally 379–382
 with KafkaTemplate 238–240
 with RabbitTemplate 228–232
 configuring message converter 231
 setting message properties 231–232
 sending resources 328–329
 sendNotification() method 440
 sendOrder() method 216
 Serializable type 110
 ServerHttpSecurity object 334
 server.port property 145, 399
 servers
 creating authorization servers 192–201
 creating RSocket 372–382
 fire-and-forget messages 378–379
 request-response 372–375
 request-stream messaging 376–377
 sending messages bidirectionally 379–382
 deleting data from 173–174
 retrieving data from 164–170
 securing API with resource server 201–204
 sending data to 170–171
 testing reactive controllers with live 324–325
 updating data on 171–173
 server.shutdown property 451
 server.ssl.key-store-password property 145
 server.ssl.key-store property 145
 @ServiceActivator annotation 253
 service activators 260–262
 @SessionAttributes annotation 38
 setAlert() method 379
 setNotificationPublisher() method 440
 setup() method 352
 shouldReturnRecentTacos() method 321
 shouldSaveAndFetchIngredients() method 346
 shouldSaveAndFetchOrders() method 360
 showDesignForm() method 37
 size parameter 179
 skip() operation 295
 @Slf4j annotation 45
 Snyder, Bruce 211
 sorting 179–180
 sort parameter 179
 source property 221
 sources() method 456
 element 39
 SpEL (Spring Expression Language) 246
 splitters 257–260
 Spring
 adding RabbitMQ to 227–228
 administering
 measures and metrics 427–431
 securing Admin server 431–433
 using Spring Boot Admin 424–427
 applications
 building and running projects 477
 initializing project from command line 472–477
 IntelliJ IDEA 463–464
 NetBeans 465–469
 Spring Tool Suite 459–462
 start.spring.io 469–472
 applying method-level security 134–136
 configuring authentication 116–124
 customizing user authentication 119–124
 in-memory user details service 118–119
 deploying
 container images 446–455
 executable JAR files 445–446
 options for 444–445
 WAR files 455–457

Spring (*continued*)

- enabling Spring Security 114–116
- initializing application 6–17
 - examining project structure 11–17
 - with Spring Tool Suite 7–11
- knowing user 136–139
- landscape 26–28
 - core Spring Framework 26
 - Spring Boot 26–27
 - Spring Cloud 28
 - Spring Data 27
 - Spring Integration and Spring Batch 27–28
 - Spring Native 28
 - Spring Security 27
- monitoring
 - creating Actuator MBeans 437–440
 - sending notifications 440–442
 - working with Actuator MBeans 435–437
- overview 4–6
- securing web requests 125–134
 - creating custom login page 128–130
 - enabling third-party authentication 131–133
 - overview 125–128
 - preventing cross-site request forgery 133–134
- setting up for Kafka 237–238
- writing application 17–26
 - building and running 21–23
 - handling web requests 18–19
 - homepages 19–20
 - Spring Boot DevTools 23–25
 - testing controller 20–21
- spring.activemq.broker-url property 213
- spring.activemq.in-memory property 214
- Spring application context 4
- spring.application.name property 426
- spring.jms.template.default-destination property 217
- Spring Batch 27–28
- Spring Boot 26–27
- Spring Boot Actuator 388–391
 - configuring base path 389–390
 - consuming endpoints 391–408
 - application information 392–395
 - runtime metrics 405–408
 - viewing application activity 403–405
 - viewing configuration details 395–403
 - customizing 408–419
 - endpoints 417–419
 - health indicators 414–415
 - registering metrics 415–417
 - the /info endpoint 408–413
 - enabling and disabling endpoints 390–391
 - securing 420–421
- spring.boot.admin.client.password property 433
- spring.boot.admin.client.url property 426
- spring.boot.admin.client.username property 433
- @SpringBootApplication annotation 15
- Spring Boot CLI (command-line interface) 27
- @SpringBootTestConfiguration-annotated class 352
- Spring Boot DevTools 23–25
 - automatic application restart 24
 - automatic browser refresh and template cache disable 24–25
 - built-in H2 console 25
- @SpringBootTest annotation 17
- Spring Cloud 28
- Spring Data 27
- Spring Data Cassandra 95–98
- spring.data.cassandra.contact-points property 97
- spring.data.cassandra.keyspace-name property 97
- spring.data.cassandra.local-datacenter property 97
- spring.data.cassandra.password property 98
- spring.data.cassandra.port property 97
- spring.data.cassandra.username property 98
- Spring Data JDBC 78–85
 - adding to build 78–79
 - annotating domain for persistence 81–83
 - preloading data with CommandLineRunner 83–85
 - repository interfaces 79–80
- Spring Data JPA 85–93
 - adding to project 85–86
 - annotating domain as entities 86–89
 - customizing repositories 90–93
 - declaring JPA repositories 89–90
- Spring Data MongoDB 106–107
- spring.data.rest.base-path property 176
- spring.datasource.data property 144
- spring.datasource.driver-class-name property 144
- spring.datasource.generate-unique-name property 65
- spring.datasource.jndi-name property 144
- spring.datasource.name property 65
- spring.datasource.schema property 144
- spring init command 475
- Spring Integration 27–28
 - creating email integration flow 267–274
 - declaring integration flow 244–251
 - configuring in Java 247–249
 - using DSL configuration 249–251
 - with XML 246–247
 - landscape 251–267
 - channel adapters 263–265
 - endpoint modules 265–267
 - filters 253–254
 - gateways 262–263
 - message channels 252–253
 - routers 256–257

- Spring Integration, landscape (*continued*)
 - service activators 260–262
 - splitters 257–260
 - transformers 254–255
- spring.kafka.bootstrap-servers property 238
- spring.kafka.template.default-topic property 240
- spring.lifecycle.timeout-per-shutdown-phase property 452
- spring.main.web-application-type property 274
- Spring Microservices in Action, 2nd Edition* (Carnell and Sanchez) 426
- Spring MVC 311
- Spring Native 28
- spring.profiles.active property 158
- spring.profiles property 157
- spring.rabbitmq.template.exchange property 230
- spring.rabbitmq.template.receive-timeout property 234
- spring.rabbitmq.template.routing-key property 230
- spring.rsocket.server.port property 374
- spring.rsocket.server.transport property 383
- Spring Security 27, 114–116
- Spring Tool Suite 7–11, 459–462
- Spring WebFlux 309–316
 - overview 310–311
 - writing reactive controllers 312–316
 - handling input reactively 315–316
 - returning single values 314
 - RxJava types 314–315
- start.spring.io 469–472
- StockQuote class 376
- streams
 - request-stream messaging 376–377
 - transforming reactive streams 295–305
 - buffering data on reactive stream 302–305
 - filtering data from reactive types 295–299
 - mapping reactive data 299–302
- String array 300
- String parameter 208, 322, 373
- String property 141
- String value 254
- String varargs 84
- submissions, form 41–49
- subscribe() method 288, 375
- Subscriber interface 282
- Subscription object 282
- systemEnvironment property source 399
- tacoById() method 314
- Taco class 34, 64, 100, 178, 341
- TacoCloudApplication class 15, 56, 456
- TacoCloudApplicationTests class 21
- tacocloud.ingredients package 402
- TacoCounter class 440
- Taco domain class 33
- Taco entity 177
- Taco entity class 340
- tacoIds property 348
- TacoMetrics bean 415
- Taco object 170, 315, 341, 439
- TacoOrderAggregateService class 348
- TacoOrder class 34, 64, 103, 137, 220, 342
- tacoOrderEmailFlow() method 270
- TacoOrder entity 177
- TacoOrder object 136, 221, 272
- taco.orders.pageSize property 150
- TacoOrder type 220
- Taco parameter 170
- TacoRepository.findAllById() method 350
- TacoRepository interface 177
- tacos.ingredients package 437
- tacos package 156
- tacos property 348
- TacoUDT class 365
- tag request attribute 407
- take() operation 291, 312
- target directory 445
- template cache disable, automatic 24–25
- testHomePage() method 21
- testing reactive controllers 320–325
 - GET requests 320–323
 - POST requests 323–324
 - with live server 324–325
- testTaco() method 321
- th:action attribute 134
- th:each attribute 39
- th:errors attribute 54
- th:field attribute 39
- th:if attribute 54
- th:src attribute 19
- third-party authentication 131–133
- thread activity 404–405
- Thymeleaf starter 25
- timeout() method 328
- toRoman() method 255
- toUser() method 124
- toUserDetails() method 336
- tracing HTTP activity 403–404
- transform() method 255
- transformation operations 295–305
 - buffering data on reactive stream 302–305
 - filtering data from reactive types 295–299
 - mapping reactive data 299–302

T

- @Table annotation 101
- Taco array 323

@Transformer annotation 255
 transformerFlow() method 255
 Transformer interface 255
 transformers 254–255
 @Transient annotation 348
 _typeId property 220

U

update() method 68
 uppercase() method 263
 UpperCaseGateway interface 263
 URI, requests with 327
 URI parameter 183
 uri tag 407
 url command-line client 188
 User class 119
 UserDetails interface 120
 UserDetails object 118, 335
 userDetailsService() method 122
 UserDetailsService bean 118, 194
 UserDetailsService interface 118
 user-info-uri property 206
 UserRepository.findByUsername() method 336
 UserRepository object 335
 users
 configuring authentication 119–124
 creating user details service 121–122
 defining user domain and persistence 119–121
 in-memory user details service 118–119
 registering users 122–124
 configuring reactive user details service 335–336
 security and knowing 136–139
 User type 120

V

@Valid annotation 52
 validating form input 49–54
 declaring validation rules 50–52
 displaying validation errors 54
 performing validation at form binding 52–54
 value attribute 39
 <version> element 286
 <version> property 447
 version property 445
 Videla, Alvaro 227
 view controllers 54–57
 view template library 57–59
 Vitale, Thomas 28

W

WAR files, building and deploying 455–457
 war plugin 457
 web APIs, reactive 333–336
 configuring reactive user details service 335–336
 configuring reactive web security 333–335
 web applications
 choosing view template library 57–59
 displaying information 30–41
 creating controller class 34–38
 designing view 38–41
 establishing domain 31–34
 processing form submission 41–49
 validating form input 49–54
 declaring validation rules 50–52
 displaying validation errors 54
 performing validation at form binding 52–54
 working with view controllers 54–57
 WebClient bean 327
 webEnvironment attribute 325
 WebMvcConfigurer interface 56
 @WebMvcTest annotation 56
 web requests 18–19, 125–134
 creating custom login page 128–130
 enabling third-party authentication 131–133
 overview 125–128
 preventing cross-site request forgery 133–134
 WebSocket 382–383
 websocket() method 383
 Web starter 25
 WebTestClient() method 323
 Williams, Jason J. W. 227
 Woolf, Bobby 244
 writeToFile() method 245
 writing applications 17–26
 building and running 21–23
 handling web requests 18–19
 homepages 19–20
 Spring Boot DevTools 23–25
 automatic application restart 24
 automatic browser refresh and template cache disable 24–25
 built-in H2 console 25
 testing controller 20–21

X

XML 246–247

Z

zip() operation 293

Spring IN ACTION Sixth Edition

Craig Walls

Spring is required knowledge for Java developers! Why? This powerful framework eliminates a lot of the tedious configuration and repetitive coding tasks, making it easy to build enterprise-ready, production-quality software. The latest updates bring huge productivity boosts to microservices, reactive development, and other modern application designs. It's no wonder over half of all Java developers use Spring.

Spring in Action, Sixth Edition is a comprehensive guide to Spring's core features, all explained in Craig Walls' famously clear style. You'll put Spring into action as you build a complete database-backed web app step-by-step. This new edition covers both Spring fundamentals and new features such as reactive flows, Kubernetes integration, and RSocket. Whether you're new to Spring or leveling up to Spring 5.3, make this classic bestseller your bible!

What's Inside

- Relational and NoSQL databases
- Integrating via RSocket and REST-based services
- Reactive programming techniques
- Deploying applications to traditional servers and containers

For beginning to intermediate Java developers.

Craig Walls is an engineer at VMware, a member of the Spring engineering team, a popular author, and a frequent conference speaker.

Register this print book to get free access to all ebook formats.
Visit <https://www.manning.com/freebook>

“The only book you'll ever need to learn and master the Spring ecosystem. This update is a must-read.”

—Pierre-Michel Ansel, 8x8

“The best resource for modern Spring development.”

—Becky Huett, Big Shovel Labs

“The definitive guide for developers wanting to build reliable, fault-tolerant, and scalable cloud-native applications using Spring.”

—David Witherspoon, Parsons

“Spring is still thriving! Get this latest edition to keep growing with it.”

—Kevin Liao, Sotheby's

“Your fast track for Spring Boot development.”

—David Torrubia Iñigo
MÁSMÓVIL Group



ISBN: 978-1-61729-757-1



9 781617 297571